

A Bounded Router-Executor Agent for CAR-bench Track 1

Lalit Chandra Routhu

rlalitchandra99@gmail.com

Abstract

We submit a bounded tool-using agent for CAR-bench Track 1, where an agent must act from the current conversation and live tool interface while remaining transparent about unavailable information [Kirmayr *et al.*, 2026]. The agent routes a request to direct execution, lightweight contract checking, or one optional reflection step before producing an A2A response. We performed limited targeted checks during development, but did not run the full Track 1 benchmark set and therefore report no Track 1 score or comparative result. Our Track 2 experiments are separate and are not used as evidence of Track 1 performance.

1 Submission Scope

CAR-bench evaluates whether tool-using agents behave consistently when a request depends on uncertain capabilities, parameters, or observations. Our Track 1 submission follows the A2A interaction setting [Agent2Agent Protocol, 2026]. The report describes the submitted approach rather than claiming that it improves benchmark performance.

The scope is intentionally narrow. We performed limited targeted, on-topic checks to exercise the request flow and selected tool-contract situations. We did not run the complete Track 1 evaluation set, repeated trials, or a full baseline comparison. Consequently, this report does not provide a task score, latency figure, cost figure, or claim of generalization. The more extensive experiments carried out for Track 2 informed development choices, but they use a separate evaluation setting and are not Track 1 results.

2 Approach

The methodology is a short routed flow shown in Figure 1. Each turn starts from the current A2A conversation, the live tool descriptions, and any observations returned earlier in that interaction. The design separates route selection, action proposal, visible-interface checks, and response emission. This is a bounded adaptation of the reasoning-and-action pattern used in tool-using language agents [Yao *et al.*, 2023; Schick *et al.*, 2023].

2.1 Context and route selection

The first stage forms the decision context from the user request, recent conversation history, live tool names and schemas, and returned observations. It does not use task identifiers, expected actions, rewards, or other evaluator-only information. The route-selection stage then chooses one of three simple behaviors. A direct route is used for an ordinary request. A guarded route is used when the answer depends on a visible tool contract or the request requires checking whether an operation is supported. A risk route is used when the candidate action should receive one additional review before it is returned.

2.2 Action proposal and visible-interface validation

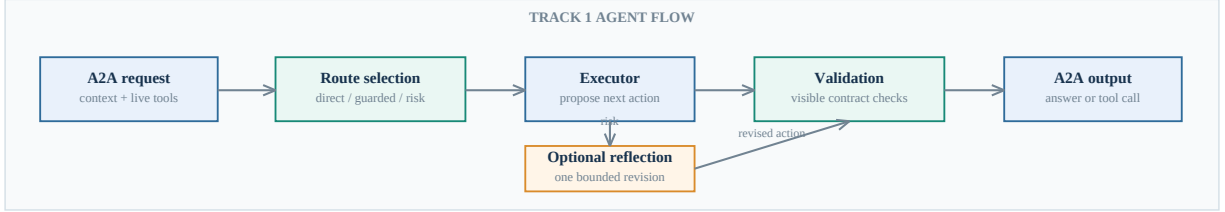
The executor receives the selected route together with the same interaction context and proposes the next agent action. That action can be a natural-language answer or a request to invoke a live tool. The validation stage then checks the proposal against the information currently exposed to the agent: the available tools, their visible argument structure, and the observations already returned. The purpose of this stage is not to infer facts that are absent from the interaction; it is to prevent an output from relying on an unavailable operation or unsupported value.

2.3 One-pass reflection and response

Only a risk-routed request enters the optional reflection stage. The stage reviews the proposed action using the same visible context and either keeps it or produces one revision. The revised action returns to validation before output. Limiting reflection to a single pass keeps the flow understandable and prevents iterative search or parallel fan-out. This use of one bounded revision is related to self-feedback and language-agent reflection methods [Madaan *et al.*, 2023; Shinn *et al.*, 2023], but this report makes no empirical claim about its effect.

After validation, the agent emits an A2A response. When the interaction contains sufficient support, this response can contain an answer or a valid tool call. When essential evidence is missing, the response explains that limitation rather than inventing a value or claiming a tool action occurred.

This design prioritizes a clear interaction boundary over a complex controller. It relies only on the information available within the active interaction.



The route and optional reflection use only the current conversation, tool schemas, and returned observations.

Figure 1: Concise Track 1 flow. The agent selects a route, proposes an action, checks it against the visible interface, and returns an A2A response. A risk-routed request may receive one optional reflection pass; there is no parallel fan-out.

3 Evaluation Status and Limitations

The available Track 1 evidence is limited to targeted development checks. Those checks helped confirm that the submission could follow the intended interaction flow, but they cannot establish benchmark reliability, tool-use quality, or the value of routing and reflection. In particular, they cannot support a comparison with other systems or an estimate of hidden-set performance.

This limitation is important when interpreting the report. Track 2 experiments were substantially more extensive, but their protocol, model configuration, and task context do not make them a substitute for a Track 1 evaluation. A full Track 1 run remains necessary to measure task success, robustness, latency, and cost.

4 Conclusion

We submit a simple bounded agent for CAR-bench Track 1. The approach routes a request through direct execution, visible-contract checking, or one optional reflection step, then returns an A2A-compatible response. The report records the approach and its limitations: only limited targeted Track 1 checks were performed, no full Track 1 benchmark experiment was run, and Track 2 results are not presented as Track 1 evidence.

References

- [Agent2Agent Protocol, 2026] Agent2Agent Protocol. Agent2Agent protocol specification, 2026.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2026.
- [Madaan *et al.*, 2023] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- [Schick *et al.*, 2023] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [Shinn *et al.*, 2023] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.